

Automação de Testes em uma Fábrica de Testes: Um Relato de Experiência

Alternative Title: Test Automation in a Test Factory: an Experience Report

Lucas Sales Vieira*
Universidade Federal do Ceará
GREat/UFC
Fortaleza, Ceará, Brasil
lucasvieira@great.ufc.br

Cayk G. Lima Barreto†
Universidade Federal do Ceará
GREat/UFC
Fortaleza, Ceará, Brasil
caykbarreto@great.ufc.br

Erick Barros dos Santos‡
Universidade Federal do Ceará
GREat/UFC
Fortaleza, Ceará, Brasil
erickbarros@great.ufc.br

Bruno Sabóia Aragão§
Universidade Federal do Ceará
GREat/UFC
Fortaleza, Ceará, Brasil
bruno@great.ufc.br

Ismayle de Sousa Santos
Universidade Federal do Ceará
GREat/UFC
Fortaleza, Ceará, Brasil
ismaylesantos@great.ufc.br

Rossana M. Castro Andrade¶
Universidade Federal do Ceará
GREat/UFC
Fortaleza, Ceará, Brasil
rossana@ufc.br

ABSTRACT

Software testing is an essential activity for software quality, since it allows the identification of bugs. However, testing is an expensive activity, because it requires time and experts. As a means of reducing the testing costs, many software companies have bet on the tests automation. As the main benefit, this automation cuts down the time spent to execute the tests. Nevertheless, there are few studies in the literature about the effort required to automate tests. This experience report describes a test automation experience in a Test Factory, describing the time spent, challenges faced, and lessons learned. In addition, this article presents a process for this activity, which was defined based on the gained experience.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**;

KEYWORDS

Software Testing, Test Automation, Test Factory

ACM Reference Format:

Lucas Sales Vieira, Cayk G. Lima Barreto, Erick Barros dos Santos, Bruno Sabóia Aragão, Ismayle de Sousa Santos, and Rossana M. Castro Andrade.

*Bolsista da Fundação Cearense de Pesquisa e Cultura

†Bolsista da Fundação Cearense de Pesquisa e Cultura

‡Bolsista de Mestrado do CNPq

§Bolsista da Fundação Cearense de Pesquisa e Cultura

¶Bolsista do CNPq de Produtividade em Desenvolvimento Tecnológico e Extensão Inovadora (DT) 2

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBSI'18, June 4–8, 2018, Caxias do Sul, Brazil

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-6559-8/18/06.

<https://doi.org/10.1145/3229345.3229356>

2018. Automação de Testes em uma Fábrica de Testes: Um Relato de Experiência: Alternative Title: Test Automation in a Test Factory: an Experience Report. In *SBSI'18: XIV Brazilian Symposium on Information Systems, June 4–8, 2018, Caxias do Sul, Brazil*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3229345.3229356>

1 INTRODUÇÃO

O teste de software é o processo de execução de um programa com a intenção de encontrar erros [12]. É uma importante atividade no ciclo de vida do desenvolvimento de um software [6], pois permite identificar se o software desenvolvido funciona corretamente e se está de acordo com os requisitos previamente definidos. Porém, os custos associados a essa atividade podem chegar a 40%-60% do valor total de um projeto [10, 16]. Esse alto custo ocorre porque o teste requer tempo, planejamento, conhecimento, infra-estrutura adequada e pessoal qualificado [12].

Portanto, a fim de minimizar os custos associados à essa atividade, indústria e academia pesquisam e avaliam técnicas e abordagens que otimizem as atividades relacionadas ao teste de software. Uma forma de reduzir os custos é através da realização de testes automatizados [10], pois assim a execução dos testes pode ser realizada rapidamente e sem a necessidade de testadores.

Todavia, existem desafios para a implantação de testes automatizados que devem ser levados em consideração, tais como [13]: (i) maior esforço por parte dos desenvolvedores para implementar casos de teste automatizados; (ii) manutenção dos *scripts* de teste; e (iii) custos iniciais mais elevados do que a execução de testes manuais.

Uma outra estratégia para reduzir os custos associados ao teste de software é a contratação de serviços de testes de organizações independentes, denominadas Fábrica de Testes [14]. Ao contratar os serviços de uma Fábrica de Testes um projeto de desenvolvimento de software pode se beneficiar de testes com alta qualidade e baixo custo, uma vez que não é necessário investir em sua própria equipe de testes [4].

Na literatura existem alguns relatos de experiência sobre a automação de testes. Esses trabalhos tem por objetivo a identificação dos

benefícios e limitações dos testes automatizados [7], a identificação de melhorias na automação dos testes [8], a realização de avaliações de ferramentas [1, 11], e a definição de como trabalhar com testes automatizados utilizando metodologias ágeis [3]. Porém, nenhum dos trabalhos encontrados na literatura descreve os esforços para a automação de testes em uma Fábrica de Testes.

Sendo assim, este relato pretende descrever a experiência na automação de testes dentro da Fábrica de Testes do GREat [4], que executa projetos de pesquisa, desenvolvimento e inovação (P&D&I) com a indústria. São descritos os desafios enfrentados, além das soluções aplicadas e lições aprendidas. Espera-se que os resultados apresentados neste artigo possam ser úteis tanto para apoiar profissionais na automação de testes, quanto para direcionar pesquisas relacionadas a área.

O restante do artigo está organizado da seguinte forma: na Seção 2 é descrito o ambiente da experiência, bem como o perfil das pessoas envolvidas e as ferramentas e tecnologias utilizadas; na Seção 3 é apresentado o esforço gasto na automação dos testes de dois software; na Seção 4 é descrito o processo de automação de testes gerado a partir da experiência da automação dos testes; na Seção 5 são descritos as lições aprendidas; na Seção 6 são discutidos os trabalhos relacionados; e, por fim, na Seção 7 são apresentadas as conclusões e trabalhos futuros.

2 AMBIENTE DA EXPERIÊNCIA

A experiência deste relato se desenvolveu no contexto de uma Fábrica de Testes de um laboratório de P&D&I. Essa fábrica é responsável pelo fornecimento de serviços de testes de software desde 2014. O seu principal serviço diz respeito a documentação e execução de casos de teste funcionais, que tem por objetivo encontrar comportamentos não desejados no sistema [12]. Até meados de 2017, todos os casos de teste eram executados manualmente. Contudo, buscando melhorar o processo de testes interno, no segundo semestre de 2017 iniciou-se as atividades do processo de automação de testes de dois software testados pela Fábrica de Testes. Por motivos de confidencialidade, os dois software relacionados a este relato de experiência serão referenciados como Software A e Software B. Ambos são sistemas *web* e possuem escopo de utilização global.

2.1 Ferramentas e Tecnologias Utilizadas na Automação

Foi escolhida a linguagem de programação Java para implementar a automação dos testes, pois era a mais familiar a todos os desenvolvedores participantes do projeto e exigiria, portanto, menos tempo para a implementação dos *scripts* necessários para a automação dos testes.

No que diz respeito as ferramentas de suporte, o *framework* escolhido para automatizar os testes dos sistemas foi o Selenium¹, cujo o foco é a automação de testes em aplicações *web*. Ele foi escolhido porque possui código-fonte aberto, uma comunidade ativa de desenvolvedores e uma documentação que facilita o acesso à informações de utilização do mesmo. Cabe ressaltar, no entanto, que essa ferramenta apresenta algumas limitações, tais como: (i) impossibilidade de interação com as janelas do sistema operacional;

(ii) incompatibilidade com algumas versões de navegadores; e (iii) ausência de relatório detalhado da execução.

Outra tecnologia utilizada foi o JUnit², um *framework* para a criação de testes para a linguagem Java que já é bem estabelecido na indústria. O JUnit foi escolhido por ser simples de usar e porque alguns dos analistas de testes da equipe já possuíam experiência com esse *framework*.

Como ambiente de desenvolvimento integrado (IDE), foi utilizado o Eclipse³, pois é uma das IDEs mais utilizadas pela comunidade de desenvolvedores Java, possuindo diversos *plugins*, além de também ser de código-fonte aberto. Finalmente, o navegador utilizado para a realização dos testes foi o Google Chrome⁴, pois é o navegador padrão utilizado pelos clientes dos Software A e B.

2.2 Padrões Utilizados

Para escolher as melhores técnicas e os padrões de projeto a serem utilizados na automação dos testes, foi realizado um projeto piloto com alguns casos de teste automatizados.

A partir das observações feitas no projeto piloto, dois padrões foram selecionados: (i) o padrão *PageObject*, que representa cada página do sistema como uma *classe* e mapeia suas operações como métodos, facilitando a realização de ações em uma determinada página *web* [9]; e (ii) o padrão *Bridge*, que foi utilizado para identificar os elementos das páginas do sistema em arquivos do tipo *properties*, e assim, cada *classe* relacionada as páginas do sistema tem um arquivo de *properties* no qual são mapeados os elementos que estão presentes na página *web* [15].

2.3 Perfil da Equipe envolvida na automação dos testes

Com relação ao Software A, a equipe de automação dos testes era composta por quatro pessoas: (i) um analista de teste, com mais de três anos de experiência em automação de testes com JUnit e Selenium; (ii) um estagiário, que não possuía conhecimentos prévios com relação a automação de testes e nem com tecnologias utilizadas para sua automação; e (iii) dois pesquisadores, que possuíam experiência com automação de testes utilizando Selenium e JUnit. É importante ainda ressaltar que somente o analista de testes possuía conhecimento prévio do Software A e já tinha realizado testes nesse sistema.

Quanto ao Software B, a equipe de automação dos testes era composta por três pessoas: (i) um estagiário, que possuía conhecimentos prévios com automação de testes; e (ii) dois pesquisadores, que também tinham experiência com automação de testes. Todos os membros da equipe tinham experiência prévia na utilização dos *frameworks* Selenium e JUnit. Além disso, os três integrantes envolvidos na automação dos testes do Software B foram os mesmos que participaram da automação dos testes do Software A. Finalmente, cabe ressaltar que nenhum deles possuía conhecimento prévio sobre o Software B.

¹<http://www.seleniumhq.org/>

²<https://junit.org/junit5/>

³<https://www.eclipse.org/>

⁴<https://www.google.com/chrome/>

3 PROJETOS DE TESTES

O foco deste relato trata da automação dos testes de dois software, aqui referenciados como Software A e Software B. O Software A é um grande projeto com um elevado número de casos de teste a serem automatizados. Por outro lado, o Software B possuía menos casos de teste, porém com operações de interface mais complexas. Nas subseções a seguir, a experiência da automação de testes nesses dois projetos é apresentada.

3.1 Software A

O projeto de automação de testes do Software A teve início no dia 26 de julho de 2017. O Software A possuía 18 cenários de teste e 1378 casos de teste documentados. Dessa forma, para priorizar o esforço inicial de automação dos testes, foi escolhido um conjunto de casos de teste que cobriam as principais funcionalidades deste software. Este conjunto contém os casos de teste considerados base deste sistema, e é composto por 632 casos (45,86% do total de casos de teste do sistema). Cabe destacar que a execução manual deste conjunto de casos de teste gasta em média 18 homens-horas⁵.

Após a definição das tecnologias a serem utilizadas no projeto (ver Seções 2.1 e 2.2), foi iniciada a implementação do código-fonte do projeto de testes. Durante essa atividade foi possível automatizar 98,1% (622/632) dos casos de teste básicos do Software A. A cobertura total na automação não foi alcançada porque 10 casos de teste não eram passíveis de automação. Esses casos dependiam de ações por parte da equipe de desenvolvimento desse software, estando fora do escopo de atuação da equipe da Fábrica de Testes.

A Tabela 1 sumariza o esforço na automação dos testes do Software A. O Ciclo 1 corresponde ao primeiro período de implementação dos testes, finalizado no dia 22 de novembro de 2017 e totalizando um esforço de 594 homens-horas.

Tabela 1: Esforço na automação dos testes do Software A

Ciclo	Descrição	Homem-Hora
1	622 CTs implementados	594
2	134 CTs corrigidos	121

Após a implementação dos testes, durante o Ciclo 1, foi feita a execução dos testes automatizados com o objetivo de verificar o funcionamento da *suite* de testes automatizada. Todos os 622 casos de teste automatizados na etapa anterior foram executados, porém apenas 50,48% (314/622) dos casos tiveram a execução bem sucedida, totalizando 4h e 12m de execução.

Realizando uma análise dos registros de execução foi possível verificar as causas para a baixa taxa de sucesso: (i) interdependência entre os casos de teste, onde a execução de um caso de teste influenciava na execução dos demais, por exemplo, criando ou removendo dados que seriam utilizados nos outros casos de teste; (ii) erros de implementação relacionados à busca de elementos de interface nas páginas do sistema, causados por caminhos (*xpaths*) incorretos nos arquivos de *properties*, assim, um erro de execução dos testes era gerado quando o código-fonte tentava acessar o elemento; e (iii) problemas com a velocidade da execução dos testes, de forma que

⁵Unidade de medida que apresenta a soma das horas aplicadas por cada pessoa de uma equipe envolvida na execução de uma determinada tarefa

o *framework* Selenium executou alguns casos de teste rapidamente fazendo com que os elementos da tela não chegassem a carregar completamente.

Após a identificação dos problemas, a etapa de ajustes, denominada na Tabela 1 de Ciclo 2, foi iniciada no dia 23 de novembro de 2017 e finalizada no dia 18 de dezembro de 2017, totalizando um esforço de 121 homens-horas. Com os ajustes necessários realizados, a *suite* de testes foi novamente executada e o tempo total de execução foi igual a 5h e 3m com uma taxa de sucesso igual a 72,35% (450/622).

Com relação aos casos de teste que falharam, a análise das execuções demonstrou que ocorreram por conta de: (i) 8,04% (50/622) foram mudanças nas páginas do sistema, onde alguns campos nas telas tiveram suas posições modificados; (ii) 7,07% (44/622) ocasionado por mal entendimento do caso de teste, em que o responsável pela implementação desenvolveu erroneamente os casos de teste; e (iii) 12,54% (78/622) relacionado à erros de execução mencionados anteriormente. Vale ainda ressaltar que durante a execução foram encontrados dois defeitos no sistema, que não tinham sido encontrados anteriormente pela equipe de testes.

Após o término da implementação e das correções nos testes automatizados do Software A, a *suite* de teste foi executada em paralelo a execução manual dos mesmos casos de teste para que fosse comparado o tempo gasto em ambas abordagens. A Figura 1 apresenta um gráfico com o tempo de execução dos testes de forma automatizada e manual. A execução automática leva cerca de 5 horas para todos os 622 casos de teste, enquanto que a execução manual dos mesmos testes precisa de 18 homens-horas. Sendo assim, observou-se um ganho significativo no tempo de execução dos testes.

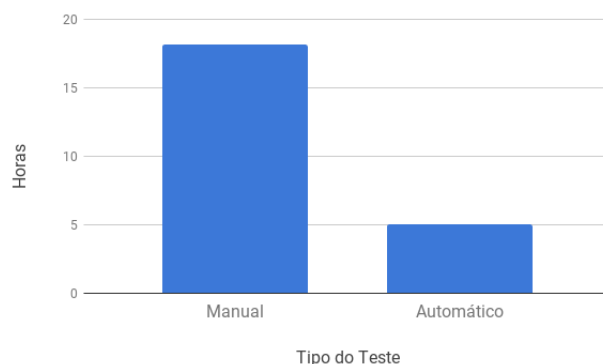


Figura 1: Tempo de Execução Manual e Automática no Software A

A Figura 2 apresenta os resultados da execução dos testes automatizados e manuais. Falhas no teste manual são referentes a problemas encontrados no software, já falhas e erros nos testes automáticos são referentes a possíveis problemas encontrados no software e erros na execução do teste automatizado, respectivamente. Com relação as falhas, os testes automatizados tem uma

porcentagem maior (15.1%) devido a alguns casos de testes implementados erroneamente, o que só foi descoberto durante a execução da *suíte* de testes.

Com base na Figura 2, percebe-se que a taxa de testes manuais que passaram é 19% maior que a dos testes automatizados, o que corresponde a um total de 118 casos de teste. Porém, vale ressaltar que mesmo adicionando o tempo necessário para executar esses 118 casos manualmente (média de quase 2 minutos por caso de teste) ao tempo de execução dos testes automatizados, seria obtido um tempo total de execução igual a 8 horas e 58 minutos. Ainda assim, esse tempo seria 9 horas menor do que a execução manual dos 622 casos de testes.

3.2 Software B

O projeto de automação do Software B teve início no dia 17 de julho de 2017. O Software B possuía 840 casos de teste. Uma abordagem semelhante a do Software A foi adotada de forma que também foi selecionado um conjunto de casos de teste que cobriam as principais funcionalidades deste projeto. Apesar da grande quantidade de casos de teste, foi determinado pelo cliente que apenas 47 casos cobriam as principais funcionalidades. Era necessário 4 homens-horas para execução desses 47 casos manualmente.

Na etapa de implementação do código-fonte do projeto de testes foi possível automatizar 84,33%(30/47) dos casos de teste das funcionalidades básicas do Software B. A cobertura total não foi possível por conta de limitações do *framework* Selenium em realizar algumas interações com a interface. Como exemplo de limitação pode ser citado a ação de arrastar elementos entre páginas do mesmo navegador, operação essa necessária para execução de alguns casos de teste. Além disso, não foi possível automatizar os casos de testes que exigiam a verificação do conteúdo de arquivos XLS e PDF baixados pelo software B.

A Tabela 2 sumariza o esforço gasto na automação do Software B. O Ciclo 1 corresponde a implementação e a primeira execução de todos os casos de teste automatizados para verificar o correto funcionamento da *suíte* de testes, totalizando um esforço de 86 homens-horas. O tempo total de execução destes casos foi de 29 minutos, onde 87,5% (24/30) obtiveram êxito em sua execução. Pelos registros de execução foi possível identificar que os demais casos falharam devido a problemas com as pré-condições necessárias para a execução de ações no Software B, e assim, foi necessário ajustar os casos de teste.

Após a identificação dos problemas e quais casos de teste deveriam ser corrigidos, a etapa de ajustes (Clico 2) foi iniciada no dia 25 de setembro de 2017 e finalizada no dia 5 de outubro de 2017, totalizando um esforço de 37 homens-horas. Após realizados os ajustes necessários nos 6 casos falhos, que tiveram um esforço de 37 homens-horas para serem corrigidos, os testes automatizados do Software B foram executados novamente. Porém, foi decidido que apenas os casos de teste que falharam teriam sua execução repetida. Dentre os casos corrigidos, 100% (6/6) obtiveram êxito em sua nova execução.

É importante ressaltar que a equipe não tinha conhecimento prévio do Software B. Além disso, os casos de testes possuíam fluxos de operações complexos, necessitando de uma quantidade elevada de horas para serem concluídos. Adicionalmente, alguns

Tabela 2: Esforço para a Automação dos Testes do Software B

Ciclo	Descrição	Homem-Hora
1	30 CTs implementados	86
2	6 CTs corrigidos	37

casos de teste exigiram esforço adicional de pesquisa a respeito do funcionamento do *framework* Selenium, devido a complexidade dos mesmos. Esses fatores impactaram na duração dessa atividade.

Após o término das correções dos testes do Software B foi realizada a execução do mesmo e seu tempo foi comparado com o tempo de execução dos manual. O gráfico da Figura 3 compara os tempos de execução manual e automatizado dos 30 casos de teste selecionados para automação.

Como apresentado na Figura 3, a execução automática do Software B gasta em torno de 29 minutos para todos os 30 casos de teste, enquanto a execução do manual dos mesmos casos precisam de 2 homens-horas para serem executados. É importante salientar que apesar das funcionalidades testadas terem sido as mesmas, não foi gerada uma imagem mostrando a proporção de erros encontrados pelos testes automatizados e manuais porque essas execuções foram realizadas em diferentes *releases* do software B.

4 PROCESSO DE AUTOMAÇÃO DE TESTES

A partir da experiência adquirida pelos autores na modelagem interativa de processos [2] e na automação de testes, relatada na Seção 3, foi possível elaborar um processo para guiar as atividades de automação. Este processo é ilustrado na Figura 4 e é constituído por duas atividades e três subprocessos. Cabe destacar que esse processo é independente do domínio da aplicação sendo testada. A seguir são descritas as atividades do processo.

- (1) **Selecionar funcionalidades cujos casos de teste podem ser automatizados:** Esta atividade recebe como entrada um documento com as funcionalidades do sistema que pretende-se automatizar os testes. Os analistas de testes devem fazer uma análise das mesmas e selecionar apenas os casos de teste que podem ser automatizados. Para definir quais casos de testes podem ser automatizados deve ser feita uma análise das possíveis restrições das ferramentas de teste utilizadas (e.g., algumas limitações do Selenium são sumarizadas na Seção 2.1), verificando se as operações necessárias nos casos de testes são suportadas. Como saída desta atividade é produzido um documento contendo as funcionalidades cujos casos de teste podem ser automatizados.
- (2) **Criar atividades de automação:** Este subprocesso recebe como entrada o documento de funcionalidades selecionadas para automação de testes, de forma que são utilizadas pelos analistas de testes para criar tarefas em um sistema de gerenciamento de projeto, como o JIRA⁶ que é usado no ambiente deste relato. Em seguida, os analistas de testes realizam as estimativas e priorizações dessas tarefas. Ao final do subprocesso é gerado um conjunto de tarefas estimadas e priorizadas.

⁶<https://www.atlassian.com/software/jira>



Figura 2: Proporção de erros Automático X Manual do Software A

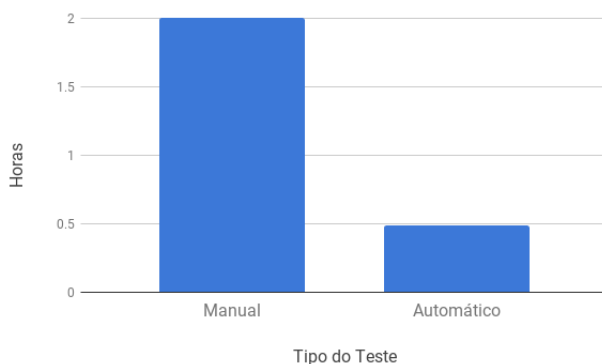


Figura 3: Tempo de Execução Manual e Automática no Software B

- (3) **Desenvolver casos de teste automatizados:** Este subprocesso recebe como entrada as tarefas do sistema JIRA, estimadas e priorizadas anteriormente. Estas tarefas serão utilizadas pelo time de analistas de testes no desenvolvimento de casos de teste automatizados. Este subprocesso inclui atividades como análise de viabilidade da automação de cada caso, implementação e ajustes de código-fonte, execução das *suites* de testes e registro de lições aprendidas sobre a implementação. Este subprocesso é repetido até que todas as funcionalidades selecionadas sejam automatizadas. Ao final, são produzidos os códigos-fonte com casos de teste selecionados para automação.
- (4) **Executar os casos de teste de todas as funcionalidades:** Esta atividade recebe como entrada os códigos-fonte com todos os casos de teste automatizados e estes são executados pelos analistas de testes. Caso não ocorram problemas na execução dos *scripts* de teste, o processo chega ao fim. Por fim, é produzido como saída um relatório de execução dos testes.
- (5) **Ajustar casos de teste:** Caso na atividade anterior sejam encontrados erros na execução dos *scripts* os mesmos são

ajustados neste subprocesso. É recebido como entrada o relatório da última execução e os casos de teste automatizados para que sejam realizados os ajustes. Outras atividades deste subprocesso incluem execução dos casos após correções, registros de lições aprendidas sobre a implementação realizada e salvar casos de teste no repositório de software. Este subprocesso é repetido até que os *scripts* de automação executem sem erros, com exceção dos casos que existem defeitos no software em teste. Como saída são produzidos casos de teste ajustados para a atividade de execução dos casos.

Dentro do processo geral, o subprocesso *Desenvolvimento de Casos de Teste Automatizados* (Figura 5) é o que possui o maior nível de esforço devido a quantidade de atividades e saídas de dados. Dessa forma, este subprocesso é descrito a seguir. Os demais subprocessos tem seus detalhes omitidos por questões de espaço.

- (1) **Selecionar um caso de teste da funcionalidade:** Esta atividade recebe como entrada as tarefas do JIRA estimadas e priorizadas, de forma que deve ser selecionado um caso de teste para realizar a automação. A atividade deve ser repetida até que todos os casos de teste passíveis de automação tenham seus *scripts* implementados. O responsável é o analista de testes e o caso de testes selecionado é a saída resultante.
- (2) **Analisar viabilidade técnica de automação:** Esta atividade tem como entrada o caso de teste selecionado na etapa anterior, onde sua viabilidade técnica de automação deve ser analisada pelo analista de testes. Esta atividade não produz nenhum artefato de saída.
- (3) **Comentar as razões na tarefa:** Caso na etapa anterior seja determinada a impossibilidade da automação do caso selecionado, o analista de testes responsável deve produzir como saída um comentário na tarefa no JIRA com as razões.
- (4) **Desenvolver o caso de teste automatizado:** Esta atividade recebe como entrada o caso de teste selecionado que será utilizado pelo analista de teste para implementar o *script* de automação do mesmo. A saída é um caso de teste que pode ser executado automaticamente.
- (5) **Executar *suite* dos casos de teste:** Esta atividade recebe como entrada a *suite* de casos de teste automatizados, constituída dos casos gerados nas atividades de desenvolvimento,

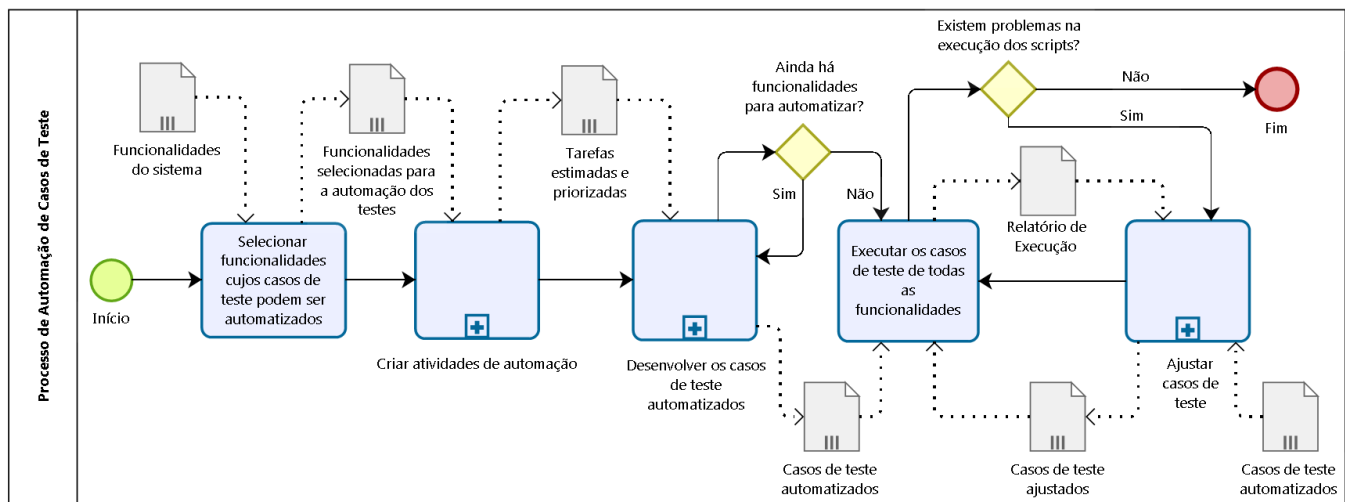


Figura 4: Processo de Automação de Casos de Teste

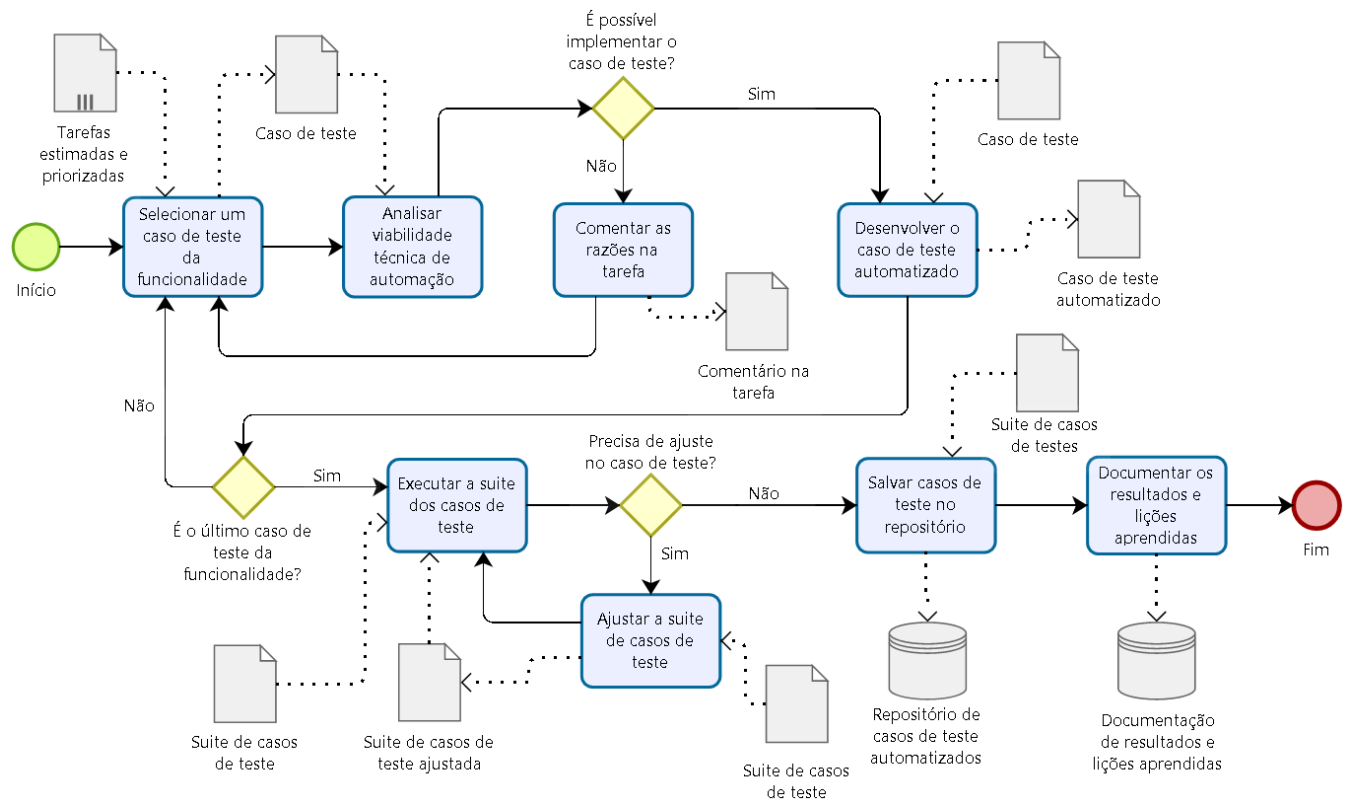


Figura 5: Subprocesso de Desenvolvimento de Casos de Teste Automatizados

para que seja executada pelo analista de teste. Não são produzidos artefatos de saída.

- (6) **Ajustar casos de teste:** Caso sejam identificados problemas na execução da atividade anterior, o analista de testes responsável deve ajustar os casos para que executem corretamente.

Esta atividade recebe como entrada a *suite* de casos de teste defeituosa e produz como saída uma *suite* de casos ajustados.

- (7) **Salvar casos de teste no repositório:** Esta atividade recebe como entrada a *suite* de casos de teste finalizada contendo os ajustes necessários e, em seguida, o analista responsável

deve submeter o código-fonte dos casos automatizados para o repositório. A saída produzida é o código-fonte dos casos devidamente versionado.

- (8) **Documentar os resultados e lições aprendidas:** Após a execução de todos os casos de teste corretamente, o analista de testes deve gerar um documento contendo os resultados e lições aprendidas durante o processo. Esta atividade não recebe entradas.

5 LIÇÕES APRENDIDAS

A automação de casos de teste de um sistema de software não é uma tarefa trivial. Wiklund e Lundqvist [17] realizaram um estudo sobre os principais impedimentos para a automação de testes. Com base na experiência descrita neste relato foi possível relacionar alguns desses impedimentos com as dificuldades encontradas na automação dos testes do Software A e Software B. A seguir são listados os problemas e lições aprendidas.

LA-1: Seguir um processo auxilia na gestão dos custos envolvidos na automação. Realizar a automação de testes é uma atividade onerosa que pode demandar longos períodos para sua conclusão. A não conclusão da automação no tempo esperado pelos *stakeholders* é citado por Wiklund e Lundqvist [17] como uma das principais causas para o abandono da automação de testes. No que se refere aos projetos desenvolvidos neste trabalho, ocorreram atrasos nas entregas inicialmente planejadas. O principal motivo para isso foi a imprecisão da estimativa no esforço necessário para a implementação e manutenção dos *scripts* de testes gerados. A solução encontrada foi incluir um tempo de manutenção nas estimativas das tarefas de automação e organizar as atividades de automação de testes (ver processo na Seção 4).

LA-2: É importante nivelar o conhecimento técnico da equipe. Embora o código-fonte necessário para a automação dos projetos deste trabalho tenham menos linhas de código e uma complexidade ciclomática inferior ao software em teste, ainda assim é exigido do responsável pela atividade alguma experiência técnica para a implementação dos *scripts*. Dessa forma, durante a execução do processo de automação foi possível identificar a necessidade dos mesmos conhecimentos técnicos mencionados por Wiklund e Lundqvist [17]: entendimento do funcionamento de sistemas *web*, *frameworks* de automação, teste de software e lógica para a implementação de *scripts*. A equipe selecionada para a automação dos testes possuía conhecimento em linguagens de programação, porém, apenas um dos integrantes tinha experiência em automação de casos de teste na indústria. Assim, identificou-se a importância da realização de treinamentos para nivelar o conhecimento da equipe, bem como a realização de reuniões de revisão para analisar os códigos-fonte produzidos e apresentar pontos de melhoria.

LA-3: Um ambiente dedicado para a execução dos testes automatizados aumenta a produtividade. A execução dos *scripts* com casos de teste automatizados no ambiente local do analista de testes demonstrou-se uma prática pouco eficiente. O mecanismo utilizado por *frameworks* de automação de interface como *Selenium* dificultam ou impedem

o uso regular do ambiente onde o teste é executado e, com isso, reduz a produtividade do responsável nas demais atividades do processo de automação. Outro fator agravante é o longo tempo necessário para a execução de grandes *suites* de casos de teste, como foi o caso do Software A. Sendo assim, identificou-se a necessidade de um ambiente exclusivamente dedicado para a execução dos testes automatizados. Esse ambiente pode ser acessado por toda a equipe, facilitando o acesso ao progresso da execução dos testes. Wiklund e Lundqvist [17] também mencionam a importância da configuração do ambiente para a realização da automação.

LA-4: Criação de uma template de desenvolvimento facilita a implementação dos scripts. Ainda que tenham sido automatizados testes para dois sistemas que possuíam propósitos diferentes para seus respectivos clientes, ambos eram sistemas *web* fazendo com que compartilhassem muitas semelhanças em seu funcionamento. Com isso, foi possível identificar similaridades no código-fonte implementado em ambos os projetos. Algumas dessas similaridades incluem: inicialização de *drivers* de automação, acesso aos elementos das páginas do sistema, e documentação de cada caso de teste na *suite*. Sendo assim, foi desenvolvido um *template* para os projetos de automação com código-fonte que poderia ser reutilizado. A utilização desse *template* implicou em algumas vantagens para o processo, tais como: aumento na velocidade de implementação, melhor organização das classes dos projetos e padronização na forma de implementação dos analistas de testes. Como resultado foi possível obter um projeto de automação mais manutenível.

LA-5: A existência de dados dedicados para a automação dos testes pode evitar falsos negativos. Após as execuções dos testes nos projetos, foi possível observar que a falta de dados dedicados para a automação de testes diminuiu a eficácia dos testes pois gerava falsos negativos, *i.e.*, casos de testes que falharam quando não deveriam. Um dos principais problemas encontrados foi a modificação do mesmo conjunto de dados em múltiplos casos de teste, fazendo com que esses dados se tornassem inválidos para os testes subsequentes, fazendo-os falhar. Outro problema encontrado foi a dificuldade de obter alguns tipos de dados específicos, os quais no sistema em teste são obtidos através de operações com tempos programados. Não foi possível para os projetos desse relato gerar um conjunto de dados de uso exclusivo para os testes automatizados. Como solução foi determinada uma ordem na execução dos casos de teste. Isso permite maior controle sobre a modificação dos dados nos casos interdependentes.

6 TRABALHOS RELACIONADOS

Na literatura é possível identificar diversos trabalhos [1, 3, 5, 7, 8, 11] relacionados ao relato de experiência sobre a implantação dos testes automatizados em uma Fábrica de Testes.

Em Collins et al. [3] foi realizado um relato mostrando as experiências e resultados da implementação de três diferentes estratégias de automação em três projetos que utilizavam o *framework* SCRUM

⁷. Com base nos resultados, os autores chegaram a conclusão de que a estratégia onde os desenvolvedores e testadores colaboravam para a automação de testes unitários e testes de sistema era a mais indicada para se trabalhar com desenvolvimento ágil.

Já em Ganesan et al. [5] é feito um relato de experiência sobre o desenvolvimento de uma técnica baseada em MTB (Model Based Testing) para testar sistemas concorrentes. Através do uso dessa técnica, o testador cria um modelo com base nos requisitos do sistema e são gerados vários casos de teste a partir do modelo. Os autores concluem que é possível, com algumas modificações, a realização dos testes em sistemas concorrentes com MTB.

No trabalho de Almenar et al. [1] foi realizado um experimento para avaliar o desempenho da ferramenta TESTAR na automação de testes em aplicações *web*. A ferramenta cria *scripts* de teste com base nas informações disponibilizadas na página do sistema. Com base nas métricas de avaliação utilizadas, os autores concluem que a ferramenta consegue realizar testes automatizados em aplicações *web*. Já em Lima et al. [11] é feito um relato sobre o uso da ferramenta SilkTest, uma IDE para a criação de testes automatizados, nos projetos de uma empresa pública. A avaliação da ferramenta foi feita através de questionários com os membros da equipe e, a partir das respostas obtidas, os autores concluíram que o SilkTest teve uma boa aceitação dentro da equipe.

Em Karhu et al. [7] são feitas observações empíricas sobre os testes de software automatizados. O objetivo desse trabalho era identificar fatores que afetam a automação de testes em diferentes tipos de organizações. Através da obtenção e análise dos dados obtidos nas empresas que participaram do estudo, os autores identificaram benefícios da automação de testes, tais como o aumento da qualidade dos testes e a execução em menor tempo. Além disso, eles também identificaram algumas limitações, dentre as quais pode ser citado custos de implementação e manutenção.

Conforme apresentado, alguns trabalhos [7, 8] da literatura exploram as vantagens e desvantagens dos testes automatizados, além das melhorias necessárias na automação de testes. Outros trabalhos tratam da automação de testes com objetivos diferentes, focando nas práticas de automação em projetos ágeis [3], criação de técnicas de teste [5], e na avaliação de ferramentas de automação de testes [1, 11]. Sendo assim, o diferencial do presente trabalho é que ele descreve os esforços relacionados à automação dos testes, mostrando os problemas e soluções encontradas durante o processo de implementação e execução dos mesmos em uma Fábrica de Testes.

7 CONCLUSÃO

O teste de software é uma atividade fundamental para garantir a qualidade de um software. No entanto, essa é uma atividade onerosa e, portanto, exige das empresas soluções para reduzir os custos relacionados. Uma das soluções utilizadas tem sido automatizar os testes, que tem como principal desafio a criação e manutenção dos *scripts* de testes. Sendo assim, apesar da automação dos testes trazer benefícios, existem custos e riscos associados.

Nesse contexto, o presente artigo apresentou um relato de experiência de automação dos testes em uma Fábrica de Testes. Os dados apresentados indicam o esforço relacionado ao processo de

automação. De maneira geral, os custos iniciais foram elevados, mas a automação dos testes resultou em benefícios, uma vez que possibilitou uma redução significativa do tempo gasto nas execuções dos testes.

Foi também descrito nesse artigo um processo de automação de testes criado a partir da experiência adquirida durante as atividades realizadas em dois projetos. Por fim, foram apresentadas cinco lições aprendidas relacionadas a execução do processo. Dessa forma, tanto o processo de automação quanto as lições aprendidas podem ser utilizadas por profissionais da indústria e academia.

Como trabalhos futuros, pretende-se relacionar o custo da manutenção do código de testes com a experiência dos profissionais envolvidos. Outra direção de pesquisa é propor um modelo para estimar os custos da automação de testes. Adicionalmente, pretende-se investigar a automação de teste de aplicações móveis para comparar os custos e desafios destes com os que foram descritos neste artigo.

REFERÊNCIAS

- [1] Francisco Almenar, Anna I Esparcia-Alcázar, Mirella Martínez, and Urko Rueda. 2016. Automated testing of web applications with TESTAR. In *International Symposium on Search Based Software Engineering*. Springer, 218–223.
- [2] Bruno Aragão, Ismayle De Sousa Santos, Tales Nogueira, Lana Mesquita, and Rossana Andrade. 2017. Modelagem Interativa de um Processo de Desenvolvimento com Base na Percepção da Equipe: Um Relato de Experiência. In *Anais do Simpósio Brasileiro de Sistemas de Informação (SBSI)*.
- [3] Eliane Collins, Arilo Dias-Neto, and Vicente F de Lucena Jr. 2012. Strategies for agile software testing automation: An industrial experience. In *Computer Software and Applications Conference Workshops (COMPSACW)*. IEEE, 440–445.
- [4] Rossana Maria de Castro Andrade, Ismayle de Sousa Santos, Valéria Lelli, Kátia Marçal de Oliveira, and Ana Regina Rocha. 2017. Software Testing Process in a Test Factory - From Ad hoc Activities to an Organizational Standard. In *19th International Conference on Enterprise Information Systems*. 132–143.
- [5] Dharmalingam Ganesan, Mikael Lindvall, Stefan Hafsteinnsson, Rance Cleaveland, Susanne L Stregge, and Walter Moleski. 2016. Experience report: Model-based test automation of a concurrent flight software bus. In *Software Reliability Engineering (ISSRE), 2016 IEEE 27th International Symposium on*. IEEE, 445–454.
- [6] Vahid Garousi and Junji Zhi. 2013. A survey of software testing practices in Canada. *Journal of Systems and Software* 86, 5 (2013), 1354–1376.
- [7] Katja Karhu, Tiina Repo, Ossi Taipale, and Kari Smolander. 2009. Empirical observations on software testing automation. In *Software Testing Verification and Validation, 2009. ICST'09. International Conference on*. IEEE, 201–209.
- [8] Jussi Kasurinen, Ossi Taipale, and Kari Smolander. 2010. Software test automation in practice: empirical observations. *Advances in Software Engineering* 2010 (2010).
- [9] Dima Kovalenko. 2014. *Selenium Design Patterns and Best Practices*. Packt Publishing Ltd.
- [10] Divya Kumar and KK Mishra. 2016. The Impacts of Test Automation on Software's Cost, Quality and Time to Market. *Procedia Computer Science* 79 (2016), 8–15.
- [11] T Lima, Ayla Dantas, and Livia MR Vasconcelos. 2012. Usando o SilkTest para automatizar testes: um Relato de Experiência. In *6TH Brazilian Workshop on Systematic and Automated Software Testing*.
- [12] Glenford J Myers, Corey Sandler, and Tom Badgett. 2011. *The art of software testing*. John Wiley & Sons.
- [13] Dudekula Mohammad Rafi, Katam Reddy Kiran Moses, Kai Petersen, and Mika V Mäntylä. 2012. Benefits and limitations of automated software testing: Systematic literature review and practitioner survey. In *Proceedings of the 7th International Workshop on Automation of Software Test*. IEEE Press, 36–42.
- [14] Ana Sanz, Javier Garcia, Javier Saldana, and Antonio Amescua. 2009. A proposal of a process model to create a Test Factory. In *Software Quality, 2009. WOSQ'09. ICSE Workshop on*. IEEE, 65–70.
- [15] Alexander Shvets, M Pavlova, and G Frey. 2015. Design Patterns explained simply. URL: <https://sourcecmaking.com> [Online] (2015).
- [16] Maneela Tuteja, Gaurav Dubey, et al. 2012. A research study on importance of testing and quality assurance in software development life cycle (SDLC) models. *International Journal of Soft Computing and Engineering* 2, 3 (2012), 251–257.
- [17] Kristian Wiklund, Sigrid Eldh, Daniel Sundmark, and Kristina Lundqvist. 2017. Impediments for software test automation: A systematic literature review. *Software Testing, Verification and Reliability* 27, 8 (2017).

⁷<https://www.scrumguides.org/>